A COMPARISON OF SKILL-WRITING STYLES · OCTOBER 2026

# Why Matt Pocock's skills look the way they do, and how to write yours like that

A side-by-side reading of **38 skills** from github.com/mattpocock/skills and your **2 skills** (`tsenta-apply`, `sai-resume-tailor`), with every number measured from the files.

**148 vs 30**
words of description loaded on **every turn**: your two skills together, against one typical Matt skill

**0/2 vs 23/38**
skills that **only you** can start by typing `/name`

**2**
pointers in your skills to a **file or command that doesn't exist** (Matt: 0 found)

## The short answer

Length is **not** the difference: your bodies run about 75 lines and Matt's median is 70. The difference is **what kind of sentence fills the lines**, and that comes from what the skills are for and how they were written.

| | MATT'S SKILLS | YOUR SKILLS |
|---|---|---|
| **What they teach** | A way of **thinking**: how to interview, debug, test. Mostly process. | A **procedure with your data**: which tool, which file, which fact. |
| **Who reads them** | Thousands of strangers, any model, any repo | You, one model, one pipeline |
| **How they were made** | Drafted, then **pruned** against a written style guide (`writing-for-agents`) and a repo rulebook | **Grown** from feedback: every rejected resume added a rule |
| **Style advice followed** | "Delete what the model already knows." | Anthropic's skill-creator: "make descriptions a little bit **pushy**" so they trigger |

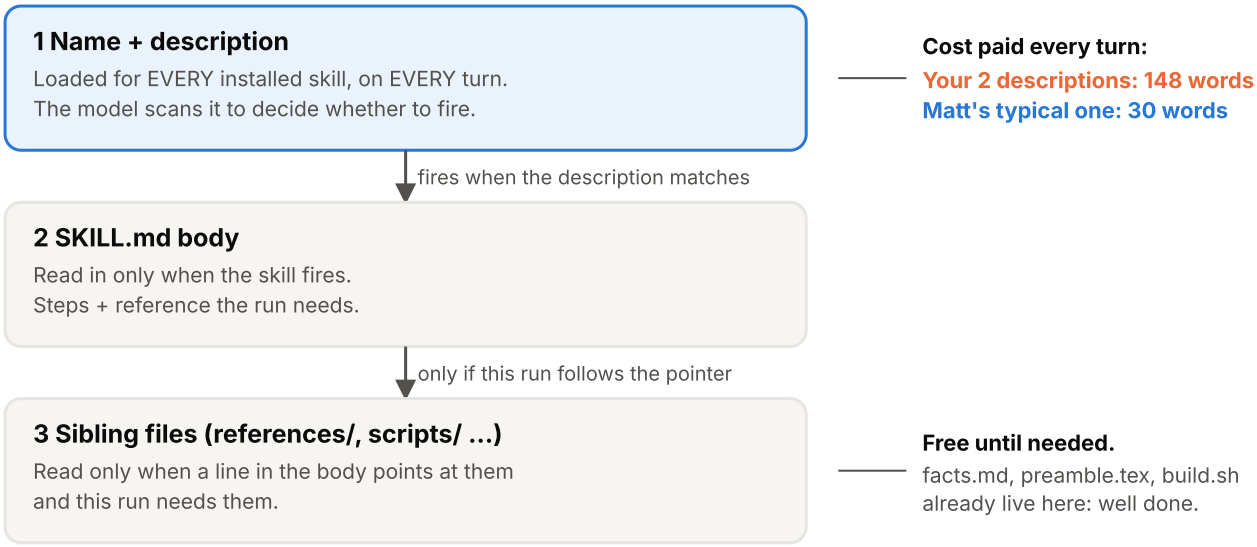
> HOW TO READ THIS REPORT
> Part 1 explains how skills load. Part 2 shows the measurements as charts. Part 3 annotates real lines from both sets. Part 4 is the glossary, Part 5 a tutorial with exercises. ● Blue is always Matt; ● orange is always you.

Sources: github.com/mattpocock/skills (MIT; `skills/productivity/writing-for-agents`, `.agents/invocation.md`, `CLAUDE.md`), `job-search/.claude/skills/tsenta-apply`, your `sai-resume-tailor` account skill, Anthropic's `skill-creator`. Every number was measured from each SKILL.md body. Longer text versions are in `job-search/docs/skill-writing/`.

PART 1 · THE MECHANICS

# How a skill is loaded, and what each level costs

A skill is a folder holding `SKILL.md` and optional sibling files. The model never reads the whole folder at once. It reads in three levels, and only the first is paid for on every turn.




**Why descriptions matter most.** Level 1 is like a table's index: scanned every time, so it should stay small. Levels 2 and 3 are like the rows and joins you only fetch on demand.

## The frontmatter fields (the block between the two `---` lines)

| FIELD | WHAT IT DOES |
|---|---|
| `name` | Lowercase with hyphens, equal to the folder name. What you type after `/`. |
| `description` | The **only part always in context**. The model reads it to decide whether to load the skill. |
| `disable-model-invocation: true` | Only a human typing `/name` can start it. No model, no other skill. |
| `argument-hint` | Placeholder shown after `/name`, e.g. "What will the next session be used for?" |
| `allowed-tools` | Tools the skill may use without a permission prompt. |

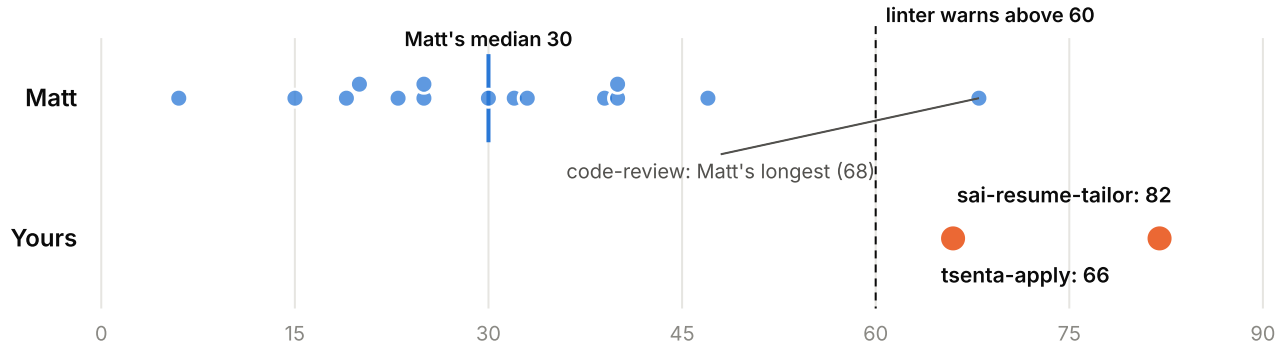
### Where a skill folder lives decides who gets it

| FOLDER | REACH |
|---|---|
| `<repo>/.claude/skills/<name>/` | Anyone working in that repo (your `tsenta-apply`) |
| `~/.claude/skills/<name>/` | You, in every repo on that machine |
| Uploaded to your claude.ai account | You in claude.ai, synced into cloud sessions (your `sai-resume-tailor`) |
| Inside a plugin | Everyone who installs it (how Matt ships his) |

PART 2 · THE MEASUREMENTS

## Difference 1: your descriptions are 2–3× longer

### Description length in words, skills the model can start




Each dot is one skill: 15 of Matt's model-invoked skills, and your 2. **Both of yours sit past the 60-word line; only 1 of Matt's 15 does.** Dashed line: where `lint_skill.py` starts warning.

Why yours grew: Anthropic's `skill-creator` warns that Claude "undertriggers" and says to make descriptions "pushy". Matt's answer is that **a sharper word triggers better than more words**. The rewrite below keeps every distinct trigger and drops the rest.

### Annotated: what to cut from the `sai-resume-tailor` description (82 → 41 words)

**BEFORE · 82 WORDS**

Tailor Sai Teja Koppolu's resume to a job description and deliver a polished PDF ~~in his own Charter-font layout~~ ❶. Use whenever Sai pastes a job description, names a role or company he is applying to, asks for a resume~~, CV, or an updated/tailored version~~ ❷, asks to change ~~bullets, projects, skills, or the layout of~~ his resume ❷, or needs application-form text (employment history, responsibilities) or a reference/outreach email. ~~Contains his verified facts, the LaTeX template, and his standing style rules.~~ ❸

**AFTER · 41 WORDS**

Tailor Sai's resume to a job posting and build the PDF. Use when Sai shares a job description or a role he is applying to, asks to change his resume, or needs application-form text or an outreach email for an application.

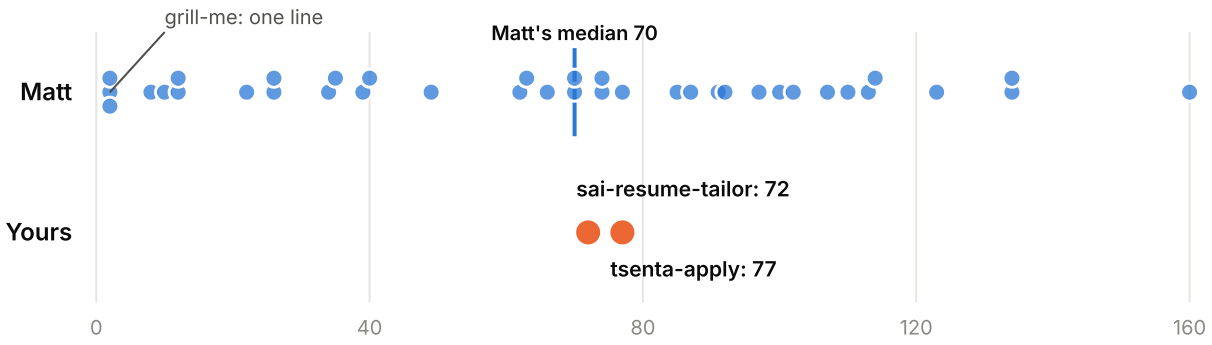
green = distinct trigger, kept · ~~red~~ = cut

| # | CUT BECAUSE | RULE BEHIND IT |
|---|---|---|
| ❶ | How it works inside | The description says **when** to fire. The body says how. |
| ❷ | Synonyms of one case | "resume, CV, updated version" is one **branch** written three times. One trigger per branch. |
| ❸ | What the folder holds | The model sees the files once the skill loads. Paying for it every turn buys nothing. |

PART 2 · THE MEASUREMENTS

## Difference 2: length isn't it. Who can start the skill is.

### SKILL.md body length in lines




**Your skills sit right on Matt's median.** His range is huge (2 to 160 lines) because he picks length by job: a skill that only hands off to another is one line long.

### Who can start the skill




Violet: disable-model-invocation: true. Gray: description is always loaded so the model can fire it.

**23 of Matt's 38 run only when he types them.** Those cost nothing per turn, because the model never sees their description.

**Model-invoked**
Description written FOR THE MODEL.
Pays CONTEXT LOAD: its words sit in every turn.
Model or you can start it.
Right for: sai-resume-tailor (tsenta-apply calls it).

**User-invoked (disable-model-invocation: true)**
One-line description FOR YOU.
Pays COGNITIVE LOAD: you must remember /name.
Only you can start it.
Consider for: tsenta-apply (it spends credits).

**The trade.** Model-invoked skills cost the model's attention every turn; user-invoked skills cost your memory. Matt spends your memory where your judgement matters, such as before spending application credits.

PART 3 · READING MATT'S STYLE UP CLOSE

# Difference 3: one word does the work of a paragraph

Below is Matt's whole `grilling` skill, shortened in two places. Highlighted words are **leading words**: short words the model already understands, defined once in bold and then reused. Green marks the **completion criterion**, the line that tells the model when it is done.

```
---
name: grilling ❶
description: Grill the user relentlessly about a plan,
  decision, or idea. Use when the user wants to
  stress-test their thinking, or uses any 'grill'
  trigger phrases. ❷
---

Interview the user relentlessly ❸ until you reach a
shared understanding. Map this as a **design tree**:
every decision branches into the decisions that
hang off it.

Work the tree in **rounds**. The **frontier** is
every decision whose prerequisites are already
settled ❹. Ask the whole frontier in one round:
number each question and give your recommended
answer. Then wait for the user's answers.

  [ format template, 10 lines ]

Finding _facts_ is your job, never the user's. ❺
... The _decisions_ are the user's.

The session is done when the frontier is empty:
every branch of the design tree visited,
nothing left silently assumed. ❻
```

❶ **No title, no overview.** The body starts with the first instruction.

❷ **The description opens on its leading word** ("Grill"), then gives two distinct triggers. 25 words.

❸ **A strong word beats a weak one.** "Relentlessly" changes behaviour; "be thorough" would be a **no-op**, because the model is already thorough-ish.

❹ **A coined term, defined once.** After this line, "frontier" means "the questions you can ask now". Every later use costs one word.

❺ **Splits the work** into what the model does (facts) and what you decide (decisions). One line, no list of examples.

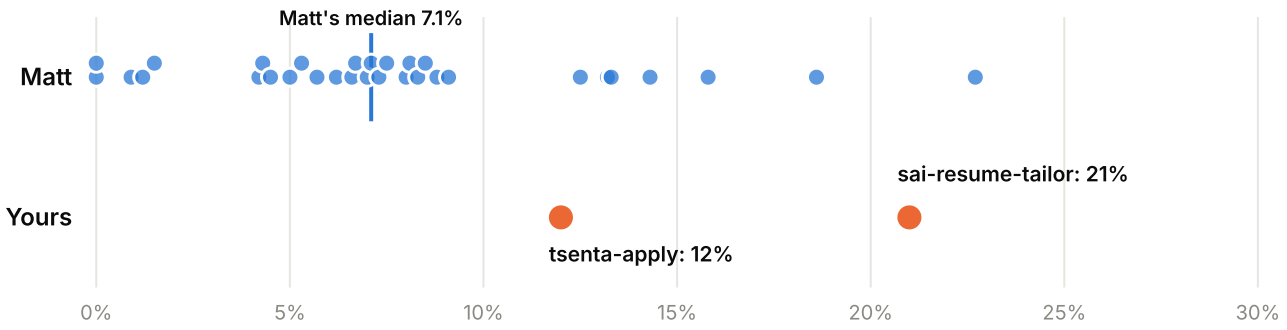
❻ **A done-line built from the leading words.** Checkable: is the frontier empty? This stops **premature completion**.

> YOUR VERSION OF THIS
> You already have one strong leading word:
> EVIDENCE MAP
> ("every bullet must come from a row of this map").
> CORE PROBLEM
> is the next candidate: you use it in steps 1 and 3 and the summary rule, but never bold it as a defined term.

PART 3 · READING THE STYLES UP CLOSE

# Difference 4: say what to do, not what to avoid

### Share of lines that say never / don't / do not / avoid




Matt's 31 skills of 20+ lines. **Your resume skill steers by prohibition 3× as often as his typical skill.**

**Why it matters:** "Don't think of an elephant" puts the elephant in your head. Naming a banned behaviour puts it in the model's context and makes it *more* available. State the target instead. Keep "never" for hard guardrails, and pair it with what to do.

| BEFORE (YOURS) | AFTER | WHY |
|---|---|---|
| Rejected: *Invented "realistic estimate" metrics or invented projects* | Every number and project comes from `facts.md` ; a bullet without a number ends on the deliverable or the decision it fed. | Names the target |
| *No line that holds only one or two words at the end of a bullet* | End every bullet on a line that is at least half full. | Checkable |
| *Do not string the posting's responsibilities together into a list of verbs* | Each summary sentence makes one claim and gives its proof. | Says what good looks like |
| *Never call apply-with-custom-resume without Sai's go-ahead for that job* | **Keep as is.** | A real guardrail, already paired with the action: get the go-ahead |

## Difference 5: calling another skill

YOURS

For each chosen job ... follow the `sai-resume-tailor` skill there.

MATT'S PATTERN

Call the Skill tool with "sai-resume-tailor", working in that folder.

Naming the tool turns a mention into an action. Matt's repo notes this raises the hit rate.

## Difference 6: every step ends in a checkable "done"

YOURS (STEP 2)

Build the evidence map in a scratch file `jd_map.md` : one row per responsibility → the strongest real fact ... or "GAP".

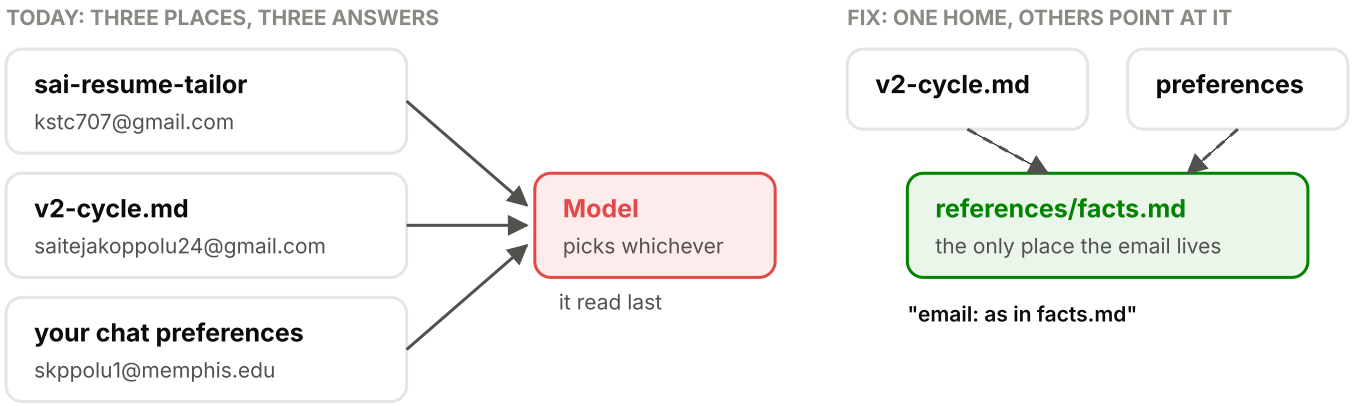
ADD A DONE-LINE

... Done when every responsibility from step 1 has a row, each holding a fact from facts.md or GAP.

Credit where due: `tsenta-apply` already does this with exit codes ("Exit 2 = blocked … fix the resume, never the rules, and rerun").

PART 3 · PROBLEMS THESE RULES WOULD HAVE CAUGHT

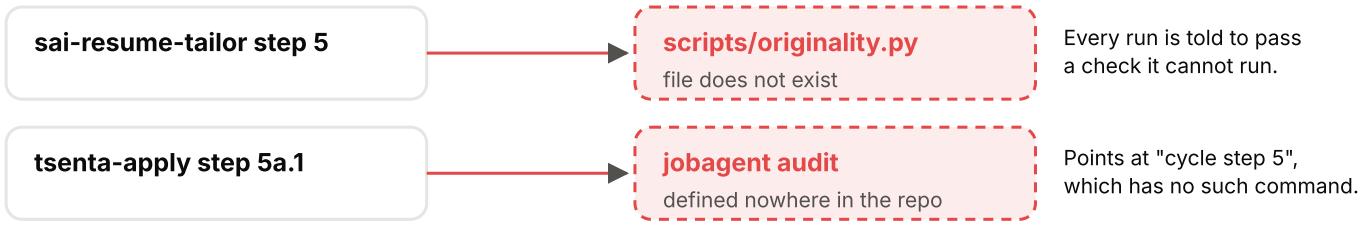
## Difference 7: one source of truth




**Same rule, three values.** Your metrics rule has the same split: the resume skill says invented metrics fail interviews, while your chat preferences ask for "a strong, believable metric" in every bullet and allow "partly aspirational" content. Pick one and point the others at it.

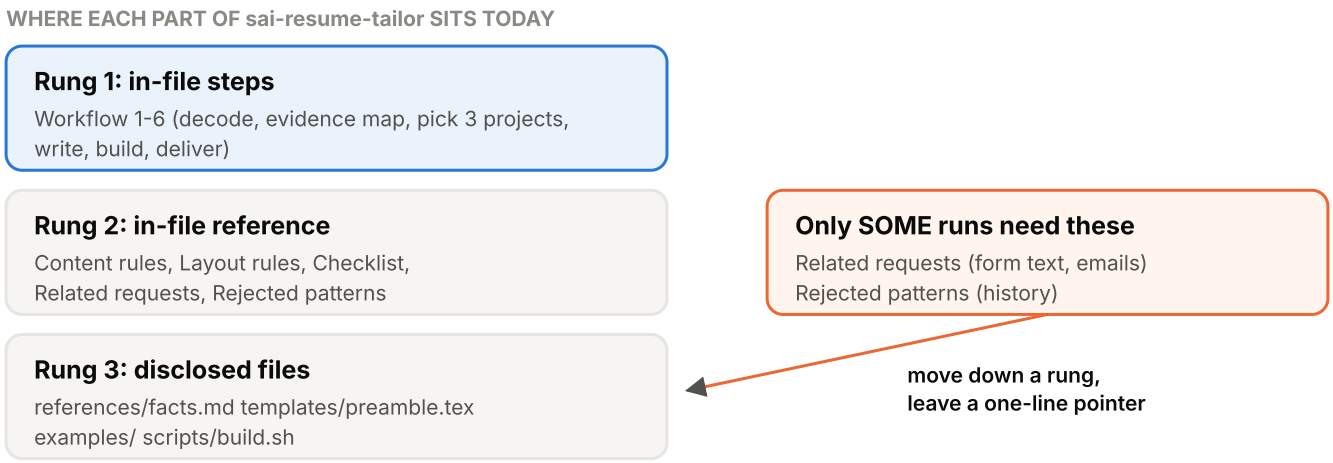
Inside `sai-resume-tailor` too: "Page 1 full" appears in Layout rules **and in two checklist items**; "never change fonts, margins, spacing" appears under Files, Layout rules and the checklist; "never cut explanation" in Content rules and Layout rules.

### Difference 8: stale pointers (Matt calls this *sediment*)




**Found by `lint_skill.py`** (the first) and by searching the repo (the second). Fix: add the missing pieces, or delete the lines that call them.

## Difference 9: load only what this run needs (*progressive disclosure*)




Test for each block: does EVERY run need it? Yes → keep in SKILL.md. Only some → move to a sibling file behind a pointer.

The same goes for `tsenta-apply`: each job takes **either** step 5a (our filler) **or** 5b (Tsenta), never both, so each could live in its own file.

PART 4 · VOCABULARY

# Every term, in plain language

**Context window**
Everything the model can see at once. Limited; attention thins as it fills.

**Context pointer**
A line naming material that isn't loaded, and when to load it. A description is one. *Like a foreign key: useless if the row is missing.*

**Branch**
A distinct case that sends a run down a different path. Synonyms are one branch.

**Context load**
Cost to the model: always-loaded words, paid every turn.

**Cognitive load**
Cost to you: remembering which skills exist and when to type them.

**Steps vs reference**
Steps are ordered actions. Reference is rules and facts looked up when needed.

**Information hierarchy**
The ladder: in-file steps → in-file reference → files behind a pointer.

**Progressive disclosure**
Moving material down the ladder. Test: does every run need it?

**Co-location**
A topic's definition, rules and caveats under one heading, not scattered.

**Sprawl**
Too long even when every line is true. Cure: disclose and split, not trim words.

**Completion criterion**
The checkable condition that ends a step. Needs *clarity* (done vs not done) and *demand* (how much it forces).

**Premature completion**
Ending a step early to reach the visible later steps. A clear criterion stops it.

**Legwork**
The digging a demanding criterion forces without writing it as a step.

**Leading word**
A short known word carrying a behaviour: *relentless, tight, red, seam, frontier, evidence map*. Define once, reuse the word.

**Negation**
Steering by "don't". Makes the banned thing more likely. State the target.

**Single source of truth**
Each rule in one place. Copies drift (your three emails).

**Cache**
A copy of something the environment already says ( `--help` , config, a script). Goes stale; point at the source.

**Sediment**
Stale lines kept because deleting feels risky. Feedback lists grow it fastest.

**No-op**
An instruction the model follows anyway ("be clear"). Test: delete it; does anything change?

**Model-invoked**
Description written for the model; model or you can start it.

**User-invoked**
`disable-model-invocation: true` ; one-line human description; only `/name` starts it.

**Router skill**
One user-invoked skill listing the others, once there are too many to remember.

**Split by sequence**
Break long steps into two skills so the model can't rush ahead.

**Split by invocation**
Make part its own model-invoked skill when it has its own trigger or another skill calls it (your resume skill, split from tsenta-apply).

> **WHAT YOU ALREADY DO BETTER THAN MATT'S AVERAGE**
> Real data in a separate file ( `facts.md` ) with a "do not claim" list · scripts that check the output ( `build.sh` , the gate's exit codes) instead of rules to remember · a feedback loop that teaches the skill. Matt's skills don't learn; yours do. Make the loop write *positive* targets, and prune old entries.

PART 5 · TUTORIAL

# Writing a skill in seven steps

The `writing-skills` skill (in `job-search/.claude/skills/`) runs exactly these steps. Each has a done-line.




## Worked example: a cover-letter skill, first draft vs pruned

**TYPICAL FIRST DRAFT**

```
description: This skill helps create professional, compelling, and tailored (1) cover letters ... a
cover letter, application letter, motivation letter, or letter of interest (2), or mentions
applying to a job (3) ...

## Overview
A cover letter is an important part of a job application... (4)

- Be professional and clear. (4)
- Don't use clichés like "I am writing to apply". (5)
- Don't invent experience. (5)
- Make sure to tailor it to the job.
- Never use the same letter twice. (6)
- Proofread carefully. (4)
```

**PRUNED**

```
description: Draft a cover letter for a specific
job posting. Use when Sai asks for a cover letter,
or asks to shorten or retone one.

Facts come only from facts.md (call the Skill tool
with "sai-resume-tailor"). Reuse this posting's
jd_map.md if it exists.

1. Name the posting's **core problem**. Done when
it names what the team needs fixed.
2. Pick 2-3 **evidence map** rows that answer it.
Done when each has a real number or deliverable.
3. One page, 3-4 paragraphs: open on the core
problem and his strongest proof; one paragraph per
row; close on the team he wants to join. Full
confidence.
4. Check: every number is in facts.md; company and
title match the posting.
```

| # | WHY IT WAS CUT |
|---|---|
| 1 | Adjectives trigger nothing. |
| 2 | Four names for one branch. |
| 3 | Too broad: would fire on every resume request. |
| 4 | No-ops: the model knows what a cover letter is and writes clearly anyway. |
| 5 | Negations that name the unwanted thing. Replaced by the opening rule and the facts pointer. |
| 6 | Same branch as "tailor it", now a checkable step 4. |

PART 5 · PRACTICE

# Exercises and cheat sheet

Rewrite each line, then check the answer on the right.

| REWRITE THIS | PROBLEM | ONE GOOD ANSWER |
|---|---|---|
| Don't forget to check the page count. | Negation, vague | Run `build.sh`; it must report 2 pages. |
| Use this skill when the user wants a resume, a CV, a résumé, or an updated resume. | Four synonyms, one branch | Use when Sai asks for a resume. |
| Be thorough when reading the job description. | Weak word (no-op) | List every responsibility in the posting; done when none is left out. |
| Make sure the output is good before sending. | No-op, no criterion | Every number appears in `facts.md`; `build.sh` reports 2 pages. |
| A 30-line Greenhouse API section in a skill every run reads, used only for Greenhouse jobs. | Should be disclosed | Move to `GREENHOUSE.md`; leave "For Greenhouse jobs, read GREENHOUSE.md." |

## Cheat sheet

| # | DO THIS | CHECK |
|---|---|---|
| 1 | Write the job in one sentence; list the **branches** | No two branches are synonyms |
| 2 | Choose model-invoked or user-invoked | Spends money or submits? Lean user-invoked |
| 3 | Description: leading word first, one trigger per branch | Under 60 words; nothing about internals |
| 4 | Body: steps that each end in "done when" | Every step has a done-line |
| 5 | Move what only some runs need into sibling files | Every pointer names a real file |
| 6 | Say what to do; keep "never" for guardrails | Each "never" is paired with an action |
| 7 | One home per rule; point at files instead of copying | Nothing stated twice |
| 8 | Delete every sentence that fails the no-op test | Delete it: does behaviour change? |
| 9 | Run it on one real task per branch | Name the failure, fix only that |
| 10 | `python3 .claude/skills/writing-skills/scripts/lint_skill.py <skill>` | Zero ERROR |

> **MATT'S OWN TEST THAT A SKILL IS GOOD**
> It gets shorter as it gets better · you can point at a leading word and see it change behaviour in more than one place · nothing is stated twice · material only one branch needs sits behind a pointer.